

Generative AI Tutorial

A Complete Guide

12 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:
<https://snehaliteng.github.io/tutorials/genai/>

1. Introduction to Generative AI

INTRODUCTION

What is Generative AI?

Generative AI refers to a class of artificial intelligence models that can generate new content — text, images, audio, video, code, and more — that resembles human-created data. Unlike discriminative models that classify or predict, generative models learn the underlying distribution of training data and sample from it to create novel outputs.

Generative vs Discriminative Models

Aspect	Discriminative Models	Generative Models
Goal	Learn decision boundary between classes	Learn the data distribution itself
Output	Class labels, probabilities	New data samples, probabilities
Examples	Logistic Regression, SVM, CNN classifiers	GANs, VAEs, GPT, Stable Diffusion
Uses	Classification, regression, detection	Content creation, synthesis, augmentation

Types of Generative Models

- **Generative Adversarial Networks (GANs)** — Two networks (generator + discriminator) compete in a minimax game
- **Variational Autoencoders (VAEs)** — Encode data to latent space, decode to reconstruct; adds probabilistic regularization
- **Autoregressive Models** — Predict next token/ pixel given previous ones (GPT, PixelCNN)
- **Flow-Based Models** — Invertible transformations for exact likelihood estimation
- **Diffusion Models** — Gradually add noise then learn to reverse the process (Stable Diffusion, DALL-E)
- **Transformer-Based** — Attention mechanisms that generate sequences (GPT-4, Claude, Gemini)

Why Now?

Three factors converged: massive datasets (the internet), powerful hardware (GPUs/TPUs), and algorithmic breakthroughs (attention, diffusion, RLHF). The result is models capable of human-like text generation, photorealistic image synthesis, and even music and video creation.

Practice Task: Use a free generative AI tool (ChatGPT, Claude, Gemini, or DALL-E) to generate something — a poem, an image, or a piece of code. Analyze the output: what's impressive? What's wrong? Compare the output with what a human might produce. Write a paragraph reflecting on the experience.

2. Evolution of Generative Models

INTRODUCTION

Timeline of Generative AI

- **1950s-1980s:** Early language models (n-grams, HMMs), Markov chains for text generation
- **1990s-2000s:** HMMs and GMMs for speech, pattern-based generation, statistical MT
- **2013:** Variational Autoencoders (VAEs) introduced by Kingma & Welling
- **2014:** Generative Adversarial Networks (GANs) introduced by Ian Goodfellow
- **2017:** Transformer architecture published in "Attention Is All You Need" (Vaswani et al.)
- **2018:** GPT-1 (117M params) — first large-scale generative pre-trained transformer
- **2019:** GPT-2 (1.5B params) — demonstrated impressive text generation; initially withheld due to safety concerns
- **2020:** GPT-3 (175B params) — few-shot learning breakthrough; DALL-E for text-to-image
- **2021:** Stable Diffusion, Midjourney, DALL-E 2 — photorealistic image generation goes mainstream
- **2022:** ChatGPT reaches 100M users in 2 months; generative AI enters mainstream consciousness
- **2023:** GPT-4, Claude, Gemini, Llama 2 — multimodal capabilities, longer context, improved reasoning
- **2024-2026:** AI agents, video generation (Sora, Veo), real-time generation, open-source models (Llama 3, Mistral)

Key Breakthroughs

- **GANs (2014)** — First framework for generating realistic synthetic data through adversarial training
- **Transformer (2017)** — Replaced RNNs with attention; enabled parallel training and much larger models
- **Diffusion Models (2020)** — Surpassed GANs for image generation with better diversity and fidelity
- **RLHF (2022)** — Aligned language models with human preferences through reinforcement learning
- **Multimodal (2023-2024)** — Models that understand and generate across text, image, audio, and video

Scaling Laws

Research shows that model performance improves predictably with increases in model size, dataset size, and compute. This empirical finding drove the "bigger is better" race, though recent focus has shifted to efficiency and alignment.

Practice Task: Create a visual timeline of generative AI milestones with at least 8 key events. For each event, note the model, parameters (if applicable), and what made it significant. Research one model from the timeline in depth and write a one-page summary of its architecture and impact.

3. ML Foundations for Generative AI

INTRODUCTION

Supervised Learning

Models learn from labeled data (input → output). For generative AI, supervised learning is used in fine-tuning — training a pre-trained model on specific (prompt, response) pairs to improve instruction following.

```
# Example: Fine-tuning a transformer for summarization
# Pseudo-code (simplified)
for batch in supervised_dataset:
    inputs = tokenize(batch["article"])
    targets = tokenize(batch["summary"])
    loss = cross_entropy(model(inputs), targets)
    optimizer.step(loss)
```

Unsupervised Learning

Models find patterns in unlabeled data. Most pre-training of generative models is unsupervised — GPT predicts the next token, BERT predicts masked tokens. This allows training on vast amounts of raw text from the internet.

Reinforcement Learning from Human Feedback (RLHF)

RLHF aligns generative models with human preferences through three stages:

1. **SFT** — Supervised fine-tuning on high-quality demonstrations

2. **Reward Modeling** — Train a reward model on human comparisons of outputs
3. **RL Optimization** — Use PPO to optimize the generative model against the reward model

```
# PPO update (simplified)
for step in rl_steps:
    outputs = model.generate(prompts)
    rewards = reward_model(outputs)
    # KL penalty to prevent too much divergence
    loss = ppo_loss(outputs, rewards, ref_logprobs)
    optimizer.step(loss)
```

Transfer Learning

Pre-train a large model on a broad dataset, then fine-tune on a specific task. This is the dominant paradigm in generative AI — train once, adapt everywhere.

Fine-tuning Strategies

- **Full Fine-tuning** — Update all parameters (expensive)
- **LoRA (Low-Rank Adaptation)** — Train small rank matrices injected into layers (efficient)
- **Adapter Layers** — Add small trainable modules between frozen layers
- **Prompt Tuning** — Learn soft prompt embeddings while keeping model frozen

Practice Task: Explain in your own words how unsupervised pre-training + supervised fine-tuning creates a capable chatbot. Use LoRA with a small open-source model (e.g., via Hugging Face PEFT) to fine-tune on a custom dataset of 20 examples. Compare training time and memory with full fine-tuning.

4. Types of Generative Models

INTRODUCTION

Overview

Model Type	Strength	Weakness	Examples
GANs	Sharp, realistic images	Mode collapse, hard to train	StyleGAN, CycleGAN, BigGAN
VAEs	Stable training, latent space	Blurry outputs	β -VAE, VQ-VAE
Autoregressive	High-quality sequential data	Slow generation (sequential)	GPT, PixelCNN, WaveNet
Flow-Based	Exact likelihood, invertible	Computationally expensive	RealNVP, Glow, Normalizing Flows
Diffusion	State-of-the-art image quality	Slow sampling (many steps)	DDPM, Stable Diffusion, DALL-E 2
Transformer	Parallel training, long context	Quadratic attention cost	GPT-4, Claude, Gemini

Autoregressive Models

Generate data one element at a time, conditioning each new element on previously generated ones. The probability of a sequence is the product of conditional probabilities: $P(x) = \prod P(x_t | x_{<t})$. GPT models are autoregressive.

Diffusion Models

Two processes: forward (gradually add Gaussian noise to data) and reverse (learn to denoise). At inference, start from pure noise and iteratively denoise to generate data. This approach powers all major text-to-image systems.

```
# Simplified diffusion training
def train_step(x_0):
    t = random_timestep()
    noise = torch.randn_like(x_0)
    x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1 - alpha_bar_t) * noise
    noise_pred = model(x_t, t)
    return MSE(noise_pred, noise)

# Sampling (reverse process)
def sample(model, steps=50):
    x = torch.randn(3, 64, 64)
    for t in reversed(range(steps)):
        noise_pred = model(x, t)
        x = denoise_step(x, noise_pred, t)
    return x
```

Which Model to Choose?

The choice depends on the task: images → diffusion or GANs; text → autoregressive transformers; structured data → VAEs or flow-based; real-time → lightweight autoregressive or GANs.

Practice Task: Create a comparison table of 5 generative model types across dimensions: training stability, inference speed, output quality, and ease of implementation. For each, identify one ideal use case and one weak area. Run a pre-trained diffusion model (e.g., Stable Diffusion via Hugging Face) and a pre-trained GPT model. Compare their outputs qualitatively.

5. Probability & Estimation for Generative AI

PROBABILITY & ESTIMATION

Probability Distributions

Generative models learn probability distributions over data. A distribution $P(x)$ assigns a probability to each possible data point x . The goal is to approximate the true data distribution with a model distribution $P_{\theta}(x)$.

- **Discrete Distributions** — Bernoulli (binary), Categorical (multi-class), Poisson (counts)
- **Continuous Distributions** — Normal/Gaussian, Uniform, Beta, Exponential
- **Mixture Models** — Combine multiple distributions (e.g., Gaussian Mixture Models)

Probability Density Functions (PDF)

For continuous data, the PDF $f(x)$ describes the relative likelihood. The integral over a range gives the probability. The Gaussian PDF is the most common: $f(x) = 1 / (\sigma\sqrt{2\pi}) \cdot \exp(-(x-\mu)^2 / (2\sigma^2))$.

Maximum Likelihood Estimation (MLE)

MLE finds parameters θ that maximize the probability of observed data. This is the core objective for most generative models. Equivalently, minimize negative log-likelihood (NLL).

```
# MLE for Gaussian distribution
def mle_gaussian(data):
    μ = mean(data)
    σ = std(data)
    return μ, σ
    # These μ, σ maximize P(data | μ, σ)

# In neural networks:
# loss = -log P_θ(data) # minimize NLL
```

KL Divergence

KL divergence measures how one distribution P differs from another Q : $D_{KL}(P || Q) = \sum P(x) \log(P(x)/Q(x))$. It's asymmetric and non-negative. Used in VAEs (KL loss) and distillation.

Bayesian Inference

Bayes' theorem updates beliefs given evidence: $P(\theta|D) = P(D|\theta) \cdot P(\theta) / P(D)$. Posterior = Likelihood $\times$ Prior / Evidence. Used in Bayesian neural networks and some generative approaches.

Note: Understanding these concepts is crucial for grasping how generative models are trained and evaluated. MLE is the most common training objective across all model types.

Practice Task: Given a dataset of 10 numbers, compute the MLE for a Gaussian distribution (mean and variance). Write Python code to compute KL divergence between two Gaussian distributions. Explain in words why minimizing KL divergence is equivalent to maximizing likelihood for generative models.

6. Generative Adversarial Networks

GENERATIVE NETWORKS

What is a GAN?

Introduced by Ian Goodfellow in 2014, GANs consist of two neural networks competing in a minimax game:

- **Generator (G)** — Takes random noise z and generates fake data $G(z)$
- **Discriminator (D)** — Attempts to distinguish real data from fake data

The generator tries to fool the discriminator, while the discriminator tries to avoid being fooled. Both improve through this adversarial process.

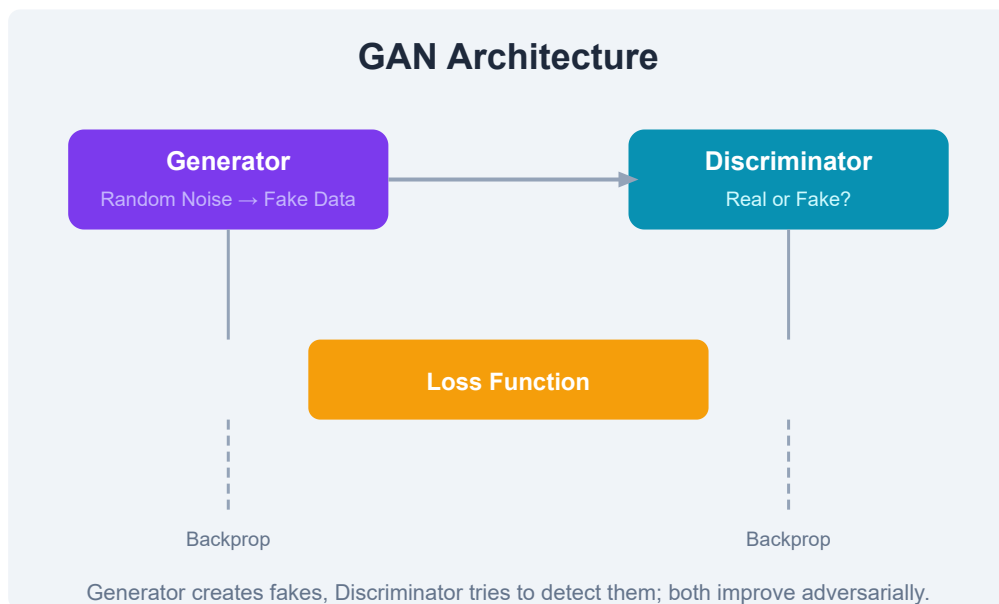


Figure: The GAN training loop — generator creates fakes, discriminator evaluates them.

Training Objective

The minimax objective: $\min_G \max_D V(D,G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$

```
# Simplified GAN training loop
for epoch in range(epochs):
    # Train discriminator
    real = sample_real_data(batch_size)
    fake = generator(noise(batch_size))
    d_loss = -mean(log(D(real)) + log(1 - D(fake)))
    d_optimizer.step(d_loss)

    # Train generator
    fake = generator(noise(batch_size))
    g_loss = -mean(log(D(fake))) # -log(D) trick
    g_optimizer.step(g_loss)
```

Important GAN Variants

- **DCGAN** — Convolutional GAN with architectural guidelines for stable training
- **Conditional GAN (cGAN)** — Both G and D receive class labels, enabling controlled generation
- **StyleGAN** — Style-based generator that separates high-level attributes from stochastic details
- **CycleGAN** — Unpaired image-to-image translation (e.g., horse → zebra)
- **Progressive GAN** — Training starts at low resolution, progressively adds layers
- **BigGAN** — Large-scale GAN for high-fidelity class-conditional image generation

Challenges in Training GANs

- **Mode Collapse** — Generator produces limited varieties of outputs
- **Non-Convergence** — Oscillations instead of stable equilibrium
- **Vanishing Gradients** — Discriminator becomes too good, generator learns nothing

- **Hyperparameter Sensitivity** — Architecture, learning rates, and initialization are delicate

Practice Task: Implement a simple GAN in PyTorch or TensorFlow to generate MNIST digits. Use a DCGAN architecture. Train for 100 epochs and generate sample images every 10 epochs. Observe how the quality improves (or fails to improve). Experiment with different latent dimensions (z). Write a paragraph explaining why mode collapse might occur.

7. Transformers in Generative AI

TRANSFORMERS IN GENERATIVE AI

The Transformer Revolution

The Transformer architecture (Vaswani et al., 2017) replaced recurrent neural networks by relying entirely on attention mechanisms. It enables parallel processing of sequences and scales to billions of parameters, forming the backbone of all major LLMs.

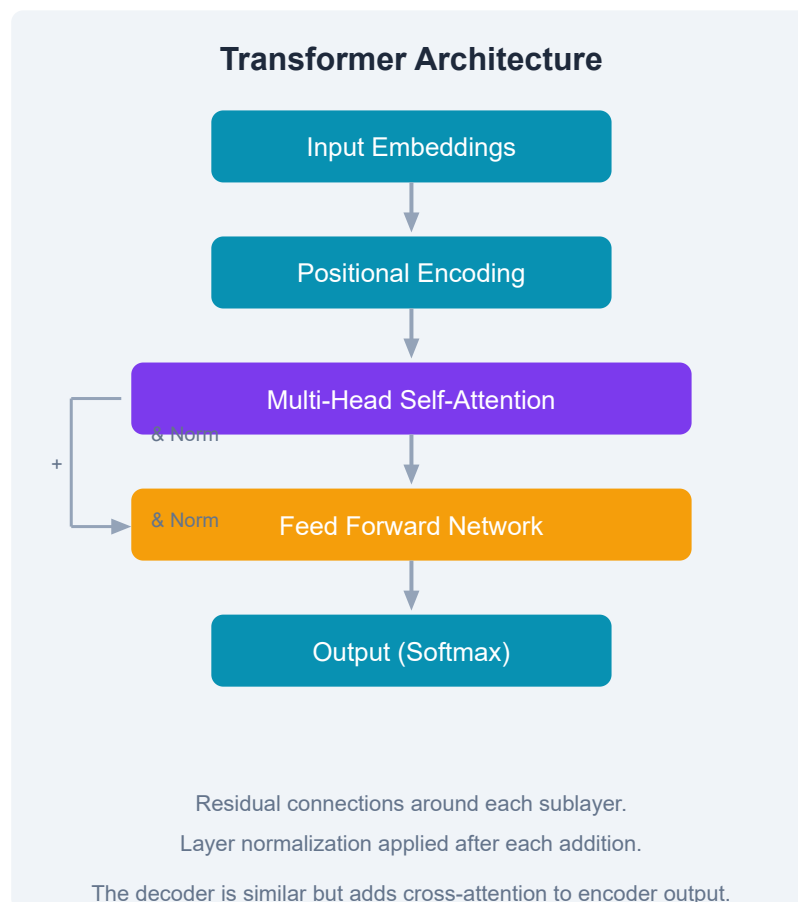


Figure: Key components of the Transformer encoder block.

Input Embeddings

Each input token is mapped to a high-dimensional vector (e.g., 768d for BERT-base, 12288d for GPT-4). Embeddings are learned during training and capture semantic relationships between tokens.

Positional Encoding

Since attention has no inherent notion of position, positional encodings are added to input embeddings. Original transformers used sinusoidal functions; learned positional embeddings are also common.

Multi-Head Attention

The core innovation. Queries (Q), Keys (K), and Values (V) are linear projections of the input. Attention weights are computed as: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d})V$. Multiple heads learn different relationship patterns.

```
# Scaled dot-product attention (single head)
def attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / sqrt(d_k)
    weights = softmax(scores, dim=-1)
    return weights @ V

# Multi-head: run attention h times in parallel, concat results
def multi_head(Q, K, V, h=8):
    heads = [attention(Q_i, K_i, V_i) for i in range(h)]
    return concat(heads)
```

Key Transformer Concepts

- **Residual Connections** — Skip connections that add input to output of each sublayer, enabling deep networks
- **Layer Normalization** — Normalize across feature dimension, stabilizes training

- **Feed Forward Network** — Two linear layers with ReLU/GELU activation applied per position
- **Causal Masking** — In decoder, mask future tokens so position t can only attend to positions $\leq t$

Decoder-Only Architecture (GPT)

GPT uses only the decoder stack with causal masking. It's autoregressive — each token is predicted from all previous tokens. The decoder consists of masked multi-head attention, then cross-attention (to encoder, if present), then FFN.

Practice Task: Implement a single layer of multi-head attention from scratch in Python with NumPy/PyTorch. Test it on a simple sequence (e.g., 5 tokens with random embeddings). Visualize the attention weights for one head as a heatmap. Explain what patterns the attention head might be capturing.

8. Autoencoders & Variational Autoencoders

AUTOENCODERS

What is an Autoencoder?

An autoencoder is a neural network that learns to copy its input to its output through a bottleneck (latent space). It consists of an encoder that compresses input to a latent representation and a decoder that reconstructs the input from the latent.

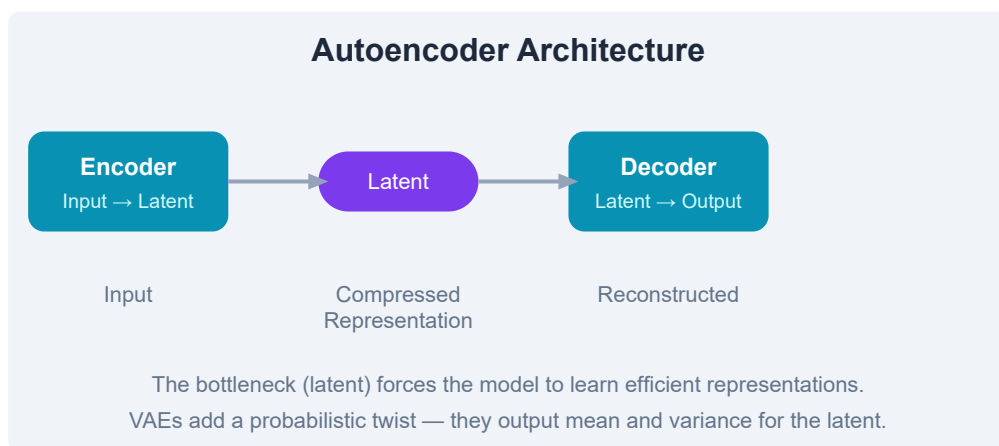


Figure: Autoencoder — input is compressed to latent, then reconstructed.

Simple Autoencoder Implementation

```
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, input_dim=784, latent_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Linear(128, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 128),
            nn.ReLU(),
            nn.Linear(128, input_dim),
            nn.Sigmoid()
        )

    def forward(self, x):
        latent = self.encoder(x)
        return self.decoder(latent)

# Loss: reconstruction error (MSE or binary cross-entropy)
```

Variational Autoencoders (VAEs)

VAEs add a probabilistic twist — instead of encoding to a single point, the encoder outputs parameters (μ , σ) of a Gaussian distribution. The decoder samples from this distribution during training. This regularizes the latent space and enables true generation.

VAE Loss

The VAE loss has two terms:

- **Reconstruction Loss** — Measures how well the decoder reconstructs the input (MSE or BCE)

- **KL Divergence** — Regularizes the latent distribution toward a standard Gaussian $N(0, I)$

```
def vae_loss(recon_x, x, μ, logvar):  
    recon = binary_cross_entropy(recon_x, x, reduction='sum')  
    kl = -0.5 * sum(1 + logvar - μ.pow(2) - logvar.exp())  
    return recon + kl  
  
# β-VAE: adds weight β on KL term for better disentanglement  
# return recon + β * kl
```

Applications of VAEs

- **Image Generation** — Generate new faces, digits, objects
- **Anomaly Detection** — High reconstruction error indicates novelty
- **Representation Learning** — Latent space captures meaningful features
- **Data Denoising** — Clean input from noisy version
- **Molecular Design** — Generate novel drug-like molecules

Practice Task: Implement a VAE in PyTorch for MNIST. Use a 2D latent space so you can visualize the latent manifold. After training, generate new digits by sampling from the prior $N(0,1)$ and decoding. Interpolate between two digit images in latent space and visualize the intermediate reconstructions. Explain what the KL term does during training.

9. ChatGPT & Large Language Models

GENERATIVE AI IN PRACTICE

What are LLMs?

Large Language Models (LLMs) are neural networks with billions of parameters trained on massive text corpora. They can generate coherent text, answer questions, translate languages, write code, and more — all by predicting the next token autoregressively.

ChatGPT Training Pipeline

1. **Pre-training** — GPT model trained on internet text (next token prediction). This is the most expensive stage, consuming thousands of GPUs for weeks.
2. **Supervised Fine-Tuning (SFT)** — Fine-tune on high-quality (prompt, response) pairs crafted by human labelers
3. **Reward Modeling** — Train a reward model on human preferences (rankings of outputs)
4. **RLHF (PPO)** — Optimize the SFT model against the reward model with a KL penalty for stability

Prompt Engineering

Techniques to get better outputs from LLMs:

- **Zero-shot** — Just ask without examples ("Summarize this article: ...")
- **Few-shot** — Provide 2-3 examples in the prompt before the real query

- **Chain-of-Thought (CoT)** — Ask the model to think step-by-step, improving reasoning
- **System Prompts** — Set the behavior, tone, and constraints at the start of the conversation
- **Role Prompting** — "Act as an expert programmer..." improves code quality

```
# Prompt template
prompt = """You are a helpful coding assistant.
Explain the following code in simple terms:

'''
def fibonacci(n):
    if n <= 1: return n
    return fibonacci(n-1) + fibonacci(n-2)
'''
"""
```

Capabilities

- Text generation, summarization, translation, question answering
- Code generation and explanation (Python, JS, SQL, etc.)
- Mathematical reasoning and problem solving
- Creative writing (poems, stories, scripts)
- Role-playing and conversation

Limitations

- **Hallucinations** — Confidently generating incorrect information
- **Context Window** — Limited attention span (4K-200K tokens depending on model)
- **Outdated Knowledge** — Training cutoff means no awareness of recent events
- **No True Understanding** — Statistical patterns, not genuine comprehension
- **Bias** — Inherits biases from training data

Practice Task: Use an LLM API (ChatGPT, Claude, or open-source via Ollama) to experiment with prompting. Compare the quality of zero-shot vs few-shot prompts for a specific task (e.g., classifying email sentiment). Try chain-of-thought prompting on a math problem. Document the prompts, outputs, and what you learned about prompt design.

10. Industry Applications of Generative AI

INDUSTRY APPLICATIONS

Manufacturing

- **Generative Design** — AI generates thousands of design variants optimized for weight, strength, and cost
- **Synthetic Data** — Generate realistic training data for computer vision models (defect detection)
- **Process Optimization** — GenAI suggests manufacturing process improvements

Software Development

- **Code Generation** — GitHub Copilot, Codeium, Cursor generate code from comments
- **Documentation** — Auto-generate API docs, READMEs, and inline comments
- **Testing** — Generate unit tests, test data, and edge cases automatically
- **Bug Fixing** — AI suggests fixes for identified bugs and vulnerabilities

Cybersecurity

- **Threat Simulation** — Generate synthetic attack patterns to test defenses
- **Phishing Detection** — GenAI generates phishing examples for training
- **Log Analysis** — Summarize and explain security logs in natural language

Marketing & Content

- **Copywriting** — Generate ad copy, email campaigns, social media posts
- **Image Generation** — Create product images, banners, and logos from text descriptions
- **Personalization** — Tailor content to individual users at scale
- **SEO** — Generate optimized meta descriptions and blog content

Healthcare

- **Drug Discovery** — Generate novel molecular structures for drug candidates
- **Medical Imaging** — Enhance low-resolution scans, generate synthetic MRI/CT data
- **Clinical Notes** — Auto-generate summaries from doctor-patient conversations

Media & Entertainment

- **Music Generation** — AI composes original music (Suno, Udio, MusicLM)
- **Video Generation** — Text-to-video (Sora, Runway, Pika)
- **Movies & Animation** — Storyboard generation, rotoscoping, VFX assistance
- **Photography** — AI upscaling, inpainting, style transfer, image restoration

Other Domains

- **Education** — Personalized lesson plans, quiz generation, tutoring
- **Cooking** — Recipe generation from available ingredients
- **Communications** — Email drafting, meeting summaries, translation
- **Legal** — Contract analysis, document summarization

Practice Task: Choose one industry from this chapter. Research three specific generative AI products or startups serving that industry. Create a use-case matrix with columns: Company, Product, Problem Solved, GenAI Technique Used, and Business Impact. Present your findings in a structured format.

11. Practice & Resources

MISCELLANEOUS TOPICS

Capstone Project Ideas

- **Image Generator** — Train a DCGAN or diffusion model on a custom dataset of 1000+ images
- **Chatbot Fine-tuning** — Fine-tune Llama 2/3 on a custom dataset using LoRA
- **Text-to-Image Pipeline** — Build a Streamlit app around Stable Diffusion with prompt engineering features
- **Music Generation** — Implement a simple RNN or transformer for MIDI music generation

Quiz Questions

1. What is the difference between a generative and discriminative model?
2. Explain the GAN training objective (minimax game).
3. How does the attention mechanism compute relationships between tokens?
4. What problem does KL divergence solve in VAEs?
5. Describe the three stages of RLHF.
6. What is mode collapse in GANs and how can it be mitigated?
7. Compare autoregressive models with diffusion models for image generation.
8. What is the purpose of positional encoding in transformers?

Ethics & Limitations

- **Misinformation** — Generative AI can create convincing fake content (deepfakes, fake news)
- **Copyright** — Legal questions around training data and generated outputs
- **Environmental Impact** — Training large models consumes significant energy
- **Job Displacement** — Automation of creative and knowledge work raises economic concerns
- **Bias Amplification** — Models may reinforce stereotypes present in training data

Future Trends

- **AI Agents** — Autonomous systems that plan, use tools, and complete complex tasks
- **Real-Time Generation** — Instant video/audio generation for live applications
- **Smaller, Efficient Models** — Distillation and quantization for edge deployment
- **Multimodal Understanding** — Seamless cross-modal generation and reasoning
- **Personalized AI** — Models that learn and adapt to individual users

Study Plan (8 Weeks)

- **Weeks 1-2:** Foundations — probability, ML basics, types of generative models
- **Weeks 3-4:** GANs and VAEs — theory, implementation, experimentation
- **Weeks 5-6:** Transformers and diffusion models — attention, GPT, Stable Diffusion
- **Weeks 7-8:** Applications, fine-tuning, ethics, capstone project

Certification Recommendations

- **DeepLearning.AI Generative AI with LLMs** — Coursera
- **Hugging Face NLP Course** — Free, practical
- **Google Cloud Generative AI** — Vertex AI
- **Fast.ai Practical Deep Learning** — Free, hands-on

Final Challenge: Build a complete generative AI project of your choice. Document the problem, approach, architecture, training process, results, and ethical considerations. Deploy a demo (Streamlit, Gradio, or Hugging Face Spaces). Share your project with the community!

12. References & Resources

REFERENCES

Generative AI Cheatsheet

Model	Type	Key Innovation	Year
VAE	Autoencoder	Probabilistic latent space + KL regularization	2013
GAN	Adversarial	Generator + Discriminator minimax game	2014
Transformer	Attention	Self-attention replaces recurrence	2017
GPT-1/2/3/4	Autoregressive	Decoder-only transformer, scaling laws	2018-2023
StyleGAN	GAN	Style-based generator, disentangled features	2019
DDPM	Diffusion	Denoising diffusion probabilistic models	2020
Stable Diffusion	Diffusion	Latent diffusion, open-source, fast	2022
ChatGPT	Autoregressive	RLHF alignment, conversational interface	2022

Glossary

Attention	Mechanism that weighs the importance of different input elements
Autoregressive	Generating one token at a time conditioned on previous tokens
Diffusion	Gradual noising → reverse denoising process
Discriminator	Network that distinguishes real from generated data (in GANs)
Embedding	Dense vector representation of a token or feature
Generator	Network that creates synthetic data (in GANs)
Hallucination	Model generating plausible but incorrect information
Latent Space	Low-dimensional representation learned by the model
LoRA	Low-Rank Adaptation — efficient fine-tuning technique
MLE	Maximum Likelihood Estimation — finding parameters that maximize data probability
Mode Collapse	GAN generator producing limited output varieties
Positional Encoding	Information added to embeddings to indicate token position
RLHF	Reinforcement Learning from Human Feedback
Softmax	Normalizes logits into a probability distribution

Useful Resources

- [Hugging Face](#) — Models, datasets, spaces, course
- [OpenAI Cookbook](#) — API examples and guides
- [Papers with Code](#) — Research papers + implementations
- [The AI Summer](#) — Tutorials and explanations
- [Lil'Log](#) — In-depth blog on generative models
- [arXiv](#) — Research preprints (search: cs.LG, cs.CL, cs.CV)

Key Research Papers

- **"Auto-Encoding Variational Bayes"** — Kingma & Welling (2013)
- **"Generative Adversarial Nets"** — Goodfellow et al. (2014)
- **"Attention Is All You Need"** — Vaswani et al. (2017)
- **"Denoising Diffusion Probabilistic Models"** — Ho et al. (2020)
- **"Training Language Models to Follow Instructions with Human Feedback"** — Ouyang et al. (2022)

Congratulations! You have completed the Generative AI Tutorial. You now have a comprehensive understanding of how generative models work, from GANs and VAEs to transformers and diffusion models. Keep experimenting and building!